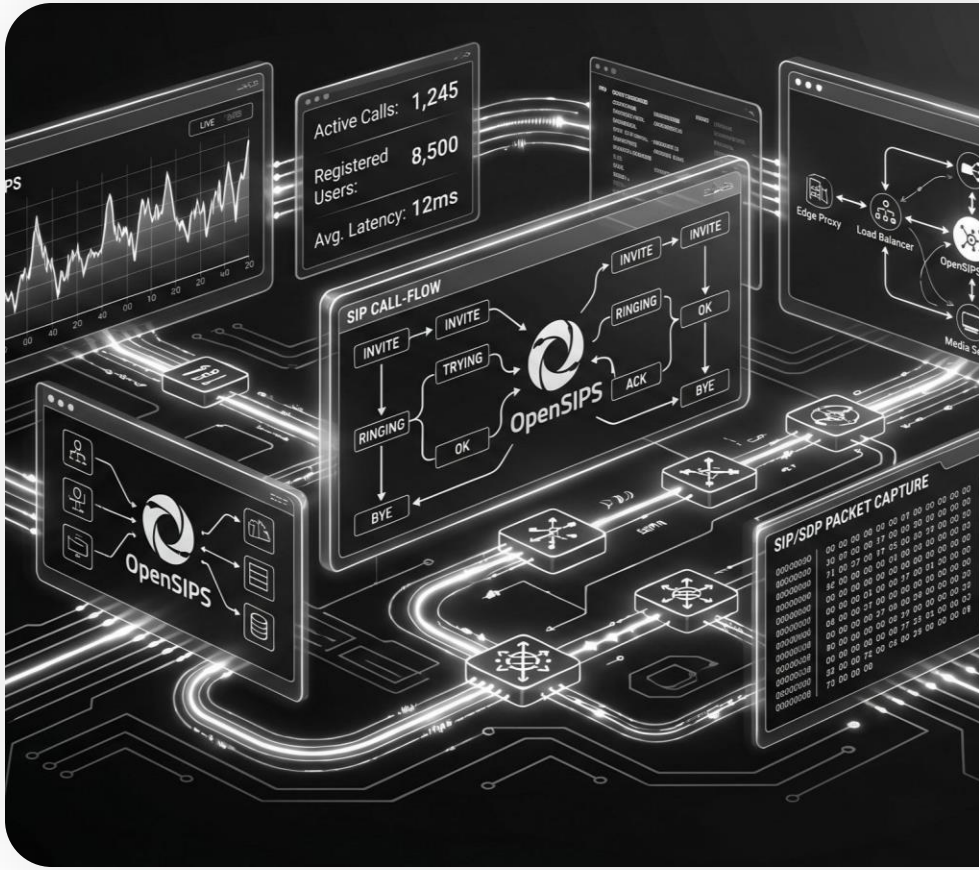
Statistics and Visualizations for OpenSIPS traffic, errors and VoIP KPIs: Know YourSelf

OpenSIPS Summit 2026



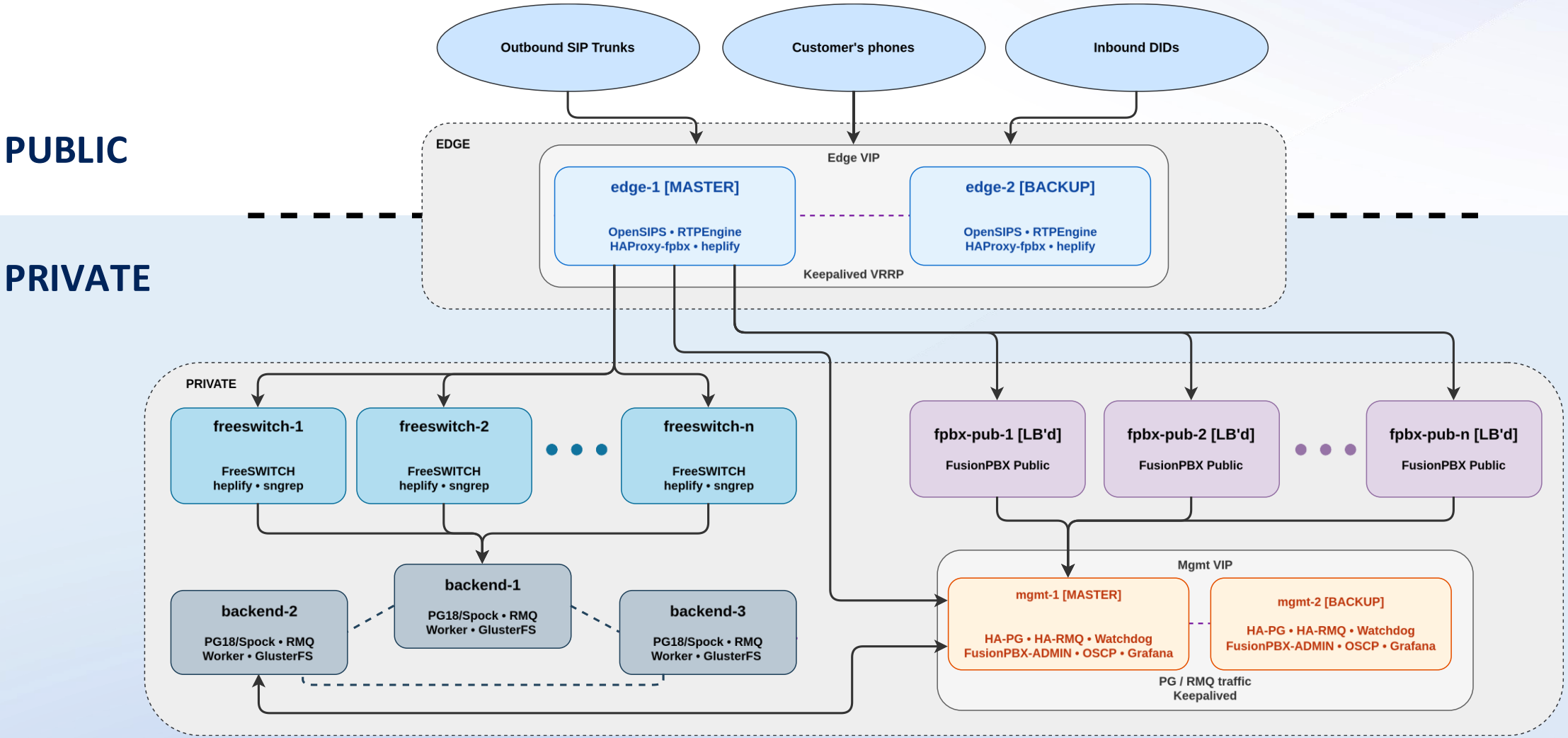
Agenda

1. Observability
2. Instrumentation: Lock-Free
3. Anomaly Detection
4. From Anomalies to Alerts
5. From Alerts to Actions

Observability

Something feels wrong. The dashboard is green.

Platform


PUBLIC
PRIVATE
Outbound SIP Trunks
Customer's phones
Inbound DIDs
EDGE
Edge VIP
edge-1 [MASTER]
OpenSIPS • RTPEngine
HAProxy-fpbx • heplify
edge-2 [BACKUP]
OpenSIPS • RTPEngine
HAProxy-fpbx • heplify
Keepalived VRRP
PRIVATE
freeswitch-1
FreeSWITCH
heplify • sngrep
freeswitch-2
FreeSWITCH
heplify • sngrep
freeswitch-n
FreeSWITCH
heplify • sngrep
fpbx-pub-1 [LB'd]
FusionPBX Public
fpbx-pub-2 [LB'd]
FusionPBX Public
fpbx-pub-n [LB'd]
FusionPBX Public
backend-1
PG18/Spock • RMQ
Worker • GlusterFS
backend-2
PG18/Spock • RMQ
Worker • GlusterFS
backend-3
PG18/Spock • RMQ
Worker • GlusterFS
Mgmt VIP
mgmt-1 [MASTER]
HA-PG • HA-RMQ • Watchdog
FusionPBX-ADMIN • OSCP • Grafana
mgmt-2 [BACKUP]
HA-PG • HA-RMQ • Watchdog
FusionPBX-ADMIN • OSCP • Grafana
PG / RMQ traffic
Keepalived

One platform, three perspectives

Business

How is each customer doing? Growing, flat, shrinking?

Are they approaching their contract's limits?

Could we upsell them something or offer the same service for cheaper?

Technical

Is the platform healthy right now?

Error rates, SIP response codes, trunk health at a glance

One tool, no hopping around to find information

In-Depth

Per-gateway, per-customer, per-route, per-source-IP

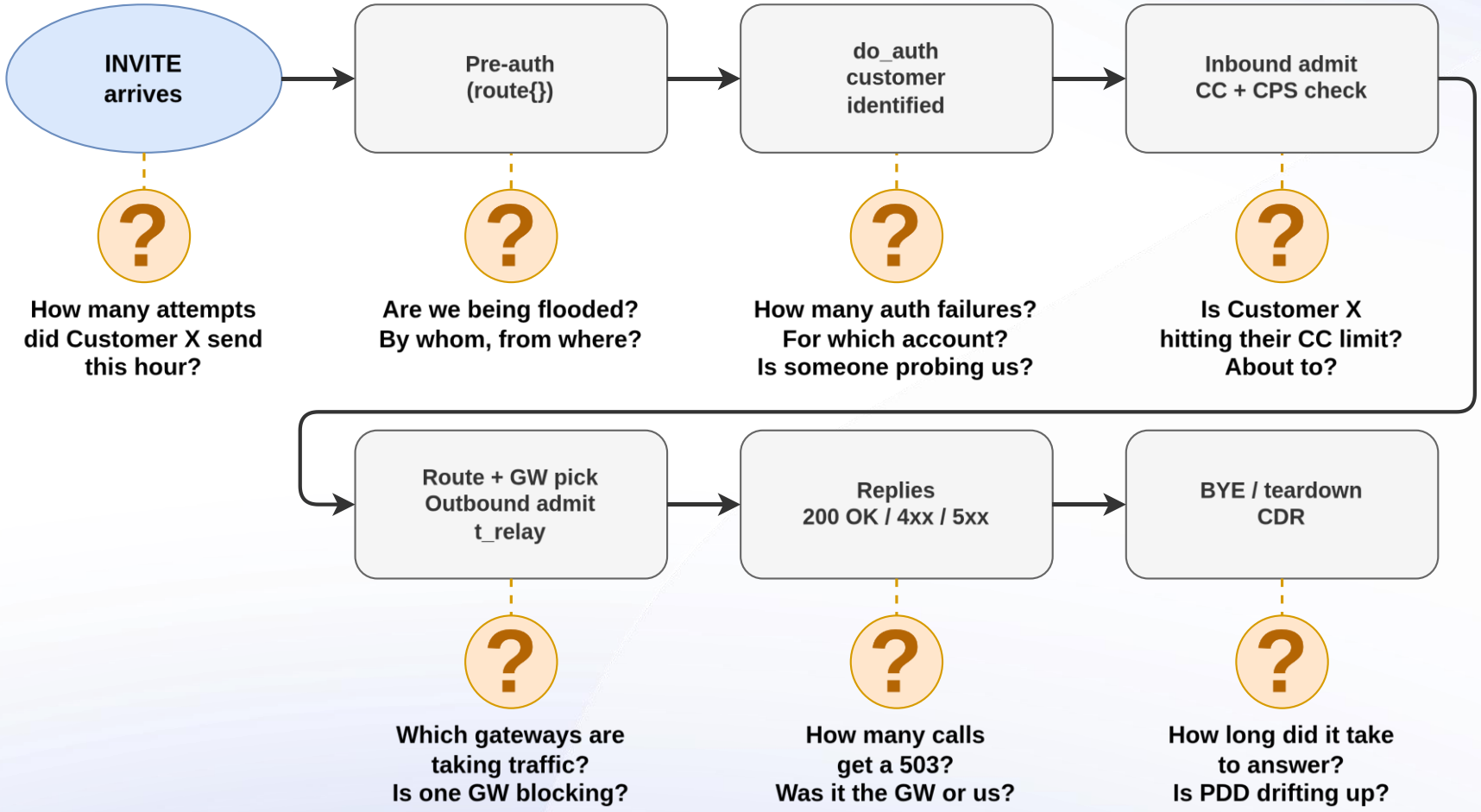
We write the dashboards – and its rules

Processing Calls

- OpenSIPS does its thing
- We want to know a lot of information about how it's doing:
  - How many calls incoming
  - How many pass auth?
  - How many of them succeed?
  - How many gateway attempts per call?

- We also want to know information grouped by certain dimensions:
  - How is a certain gateway doing?
  - How is a certain Client doing?
  - A certain route?

Processing Calls



Instrumentation: Lock-free

Nothing slows down the SIP path.

OpenSIPS: ONLY lock-free atomic writes.

- No averaging. No windowing. No ratios. Just: 'this thing happened. add x.'
- Therefore we do not grab shared memory lock for observability
- Prometheus does all time related functions: rates, averages, seasonal, bands.
- OpenSIPS never waits for observability operations.

- 100ms dashboard latency is fine. 100ms SIP Path latency is not.
- The OpenSIPS side stays lock-free, no matter how many metrics we want

Three operations. That is all the SIP path does.

Value Accumulation

For anything with a value

(MOS, PDD, Billing Duration...), we push two stats:

Route(stat_inc, $avp(…), $avp(mos))

actual MOS value

Route(stat_inc, $avp(…), 1)

+1 sample

With these we can compute averages, deviations, etc. upstream

Counter Increments

For all metrics, where only the quantity is of interest to us (drop_auth, funnel_ingested...), we push the single counter:

Route(stat_inc, $avp(…), 1)

+1 sample

Gauges Handling

For metrics where we care about the current value (concurrent calls in/out...), we create a gauge that we maniupulate:

Route(stat_inc, $avp(…), 1)

+1 concurrent

Route(stat_inc, $avp(…), -1)

-1 concurrent

OpenSIPS Configuration

```
route[stat_inc] {
#  simple lock-free abs counter increment.
# $param(1) = counter name (e.g. "GL_funnel_ingested")
# $param(2) = value (+1, -1, or any integer)
    update_stat("$param(1)", $param(2));
}
```

Lock-free atomic

```
route[no_more_gws]
{

    $avp(metric_name) = "GL_cdr_no_more_gws";
    route(stat_inc, $avp(metric_name), 1);
    $avp(metric_name) = "RU_" + $acc_extra(routeid) + "_cdr_no_more_gws";
    route(stat_inc, $avp(metric_name), 1);
    $avp(metric_name) = "CU_" + $acc_extra(accountcode) + "_cdr_no_more_gws";
    route(stat_inc, $avp(metric_name), 1);
    $avp(metric_name) = "AS_" + $avp(which_asterisk) + "_cdr_no_more_gws";
    route(stat_inc, $avp(metric_name), 1);
```

Calls to it follow the same format, with multiple dimensions per metric:

- Global
- Gateway
- Customer
- Route
- Incoming IP

Everything else runs on top - in Prometheus

Metric naming encodes dimensions:

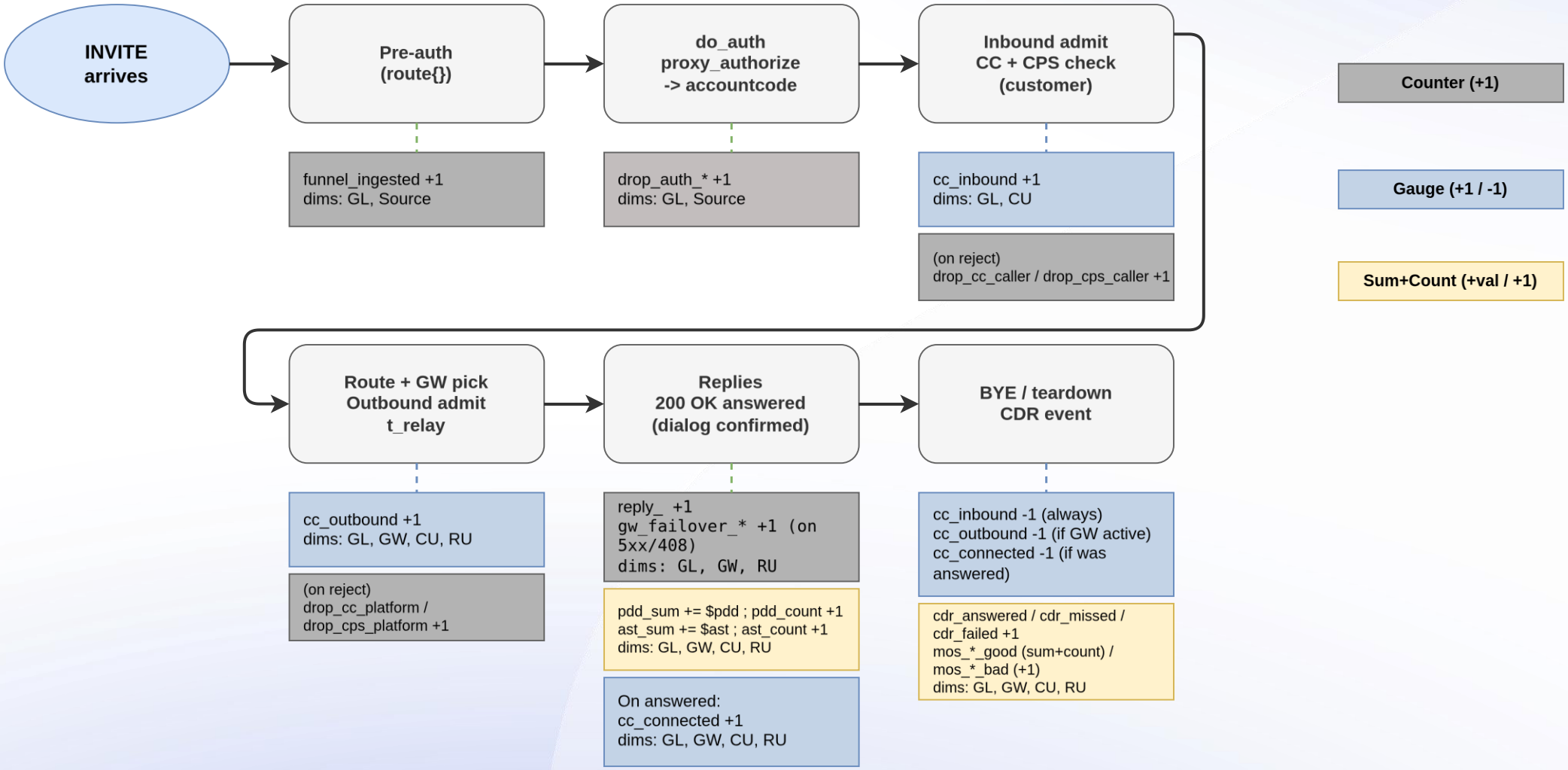
- opensips_{GL|GW|CU|RU|AS}_<name>
- GL=platform, GW=gateway, CU=customer, RU=route, AS=Incoming IP

- One counter per entity per metric. Prometheus scrapes OpenSIPS.
- label_replace() extracts the entity back out into a label.
- 50k active series. All generated from simple atomic values.

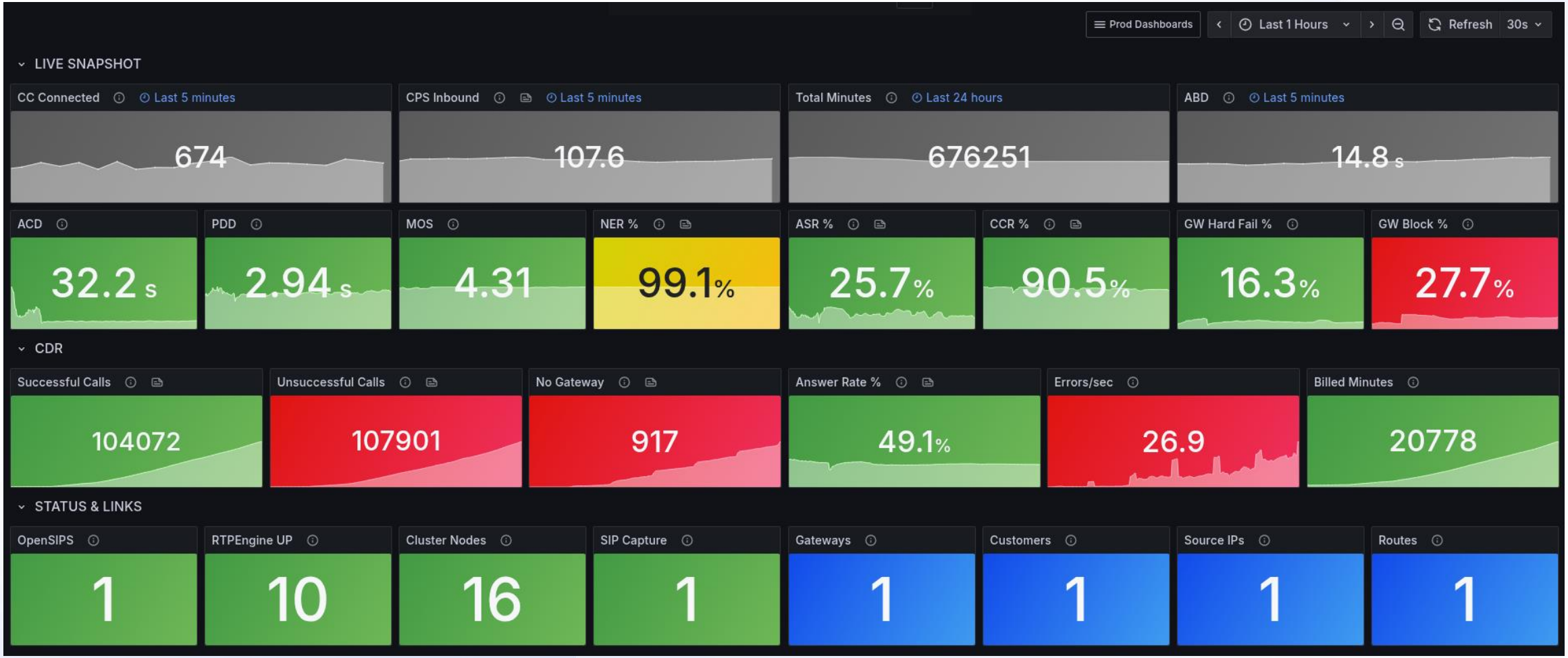
All the math lives in Prometheus (PromQL) .

- Averages, Statistical Computations, Confidence Bands are all done in PromQL.

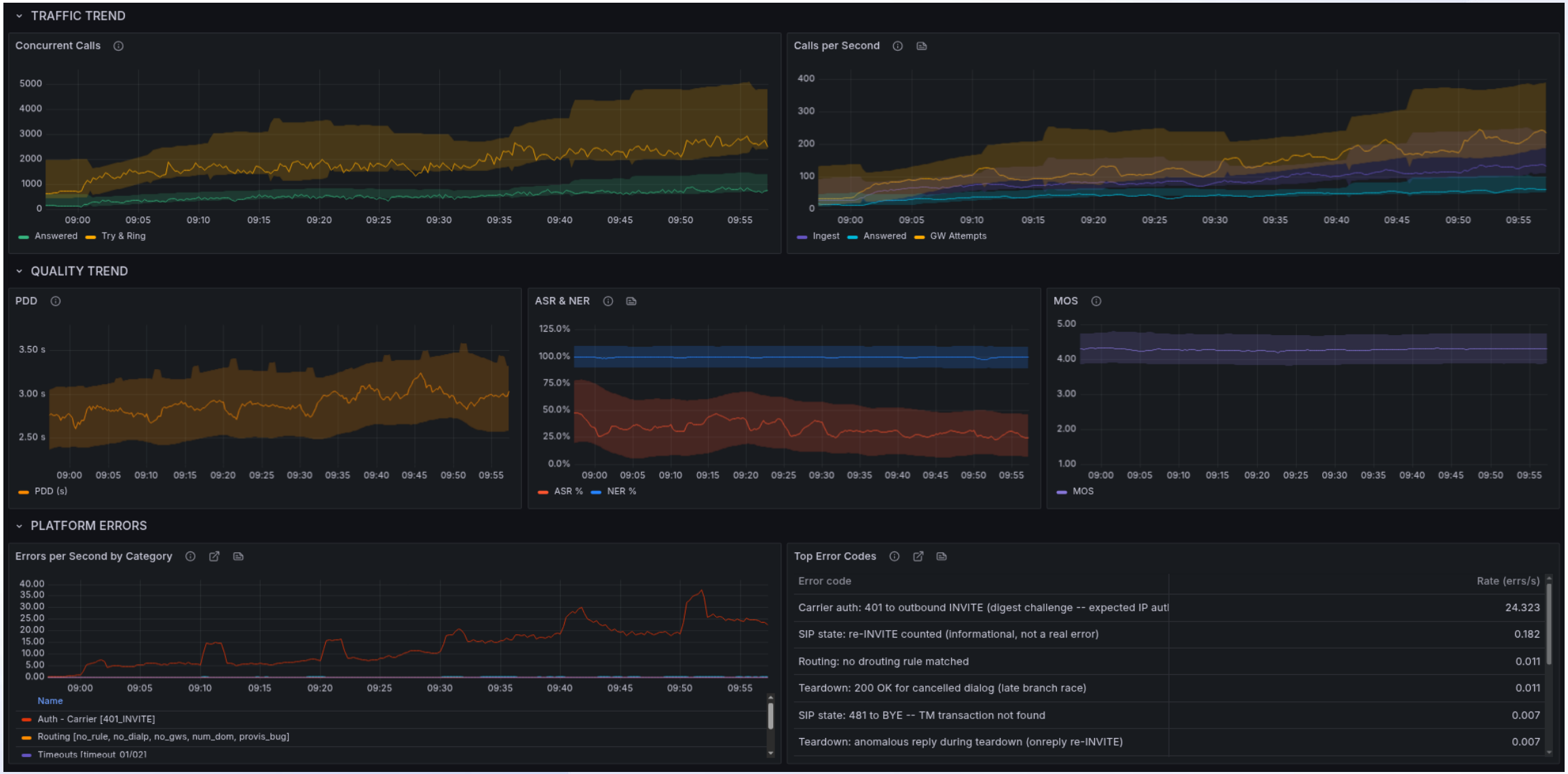
Processing Calls



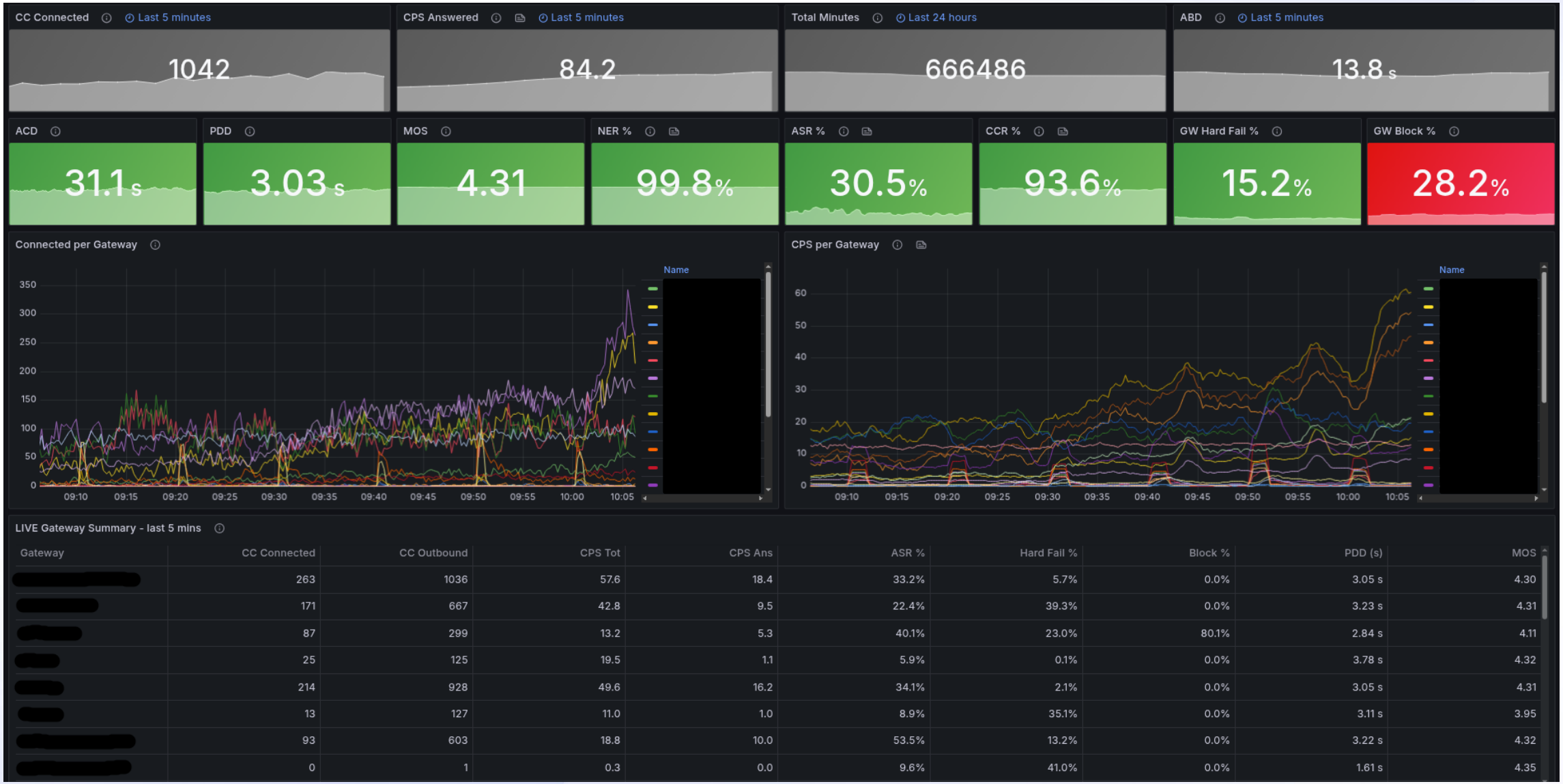
Main Dashboard - 1



Main Dashboard –2

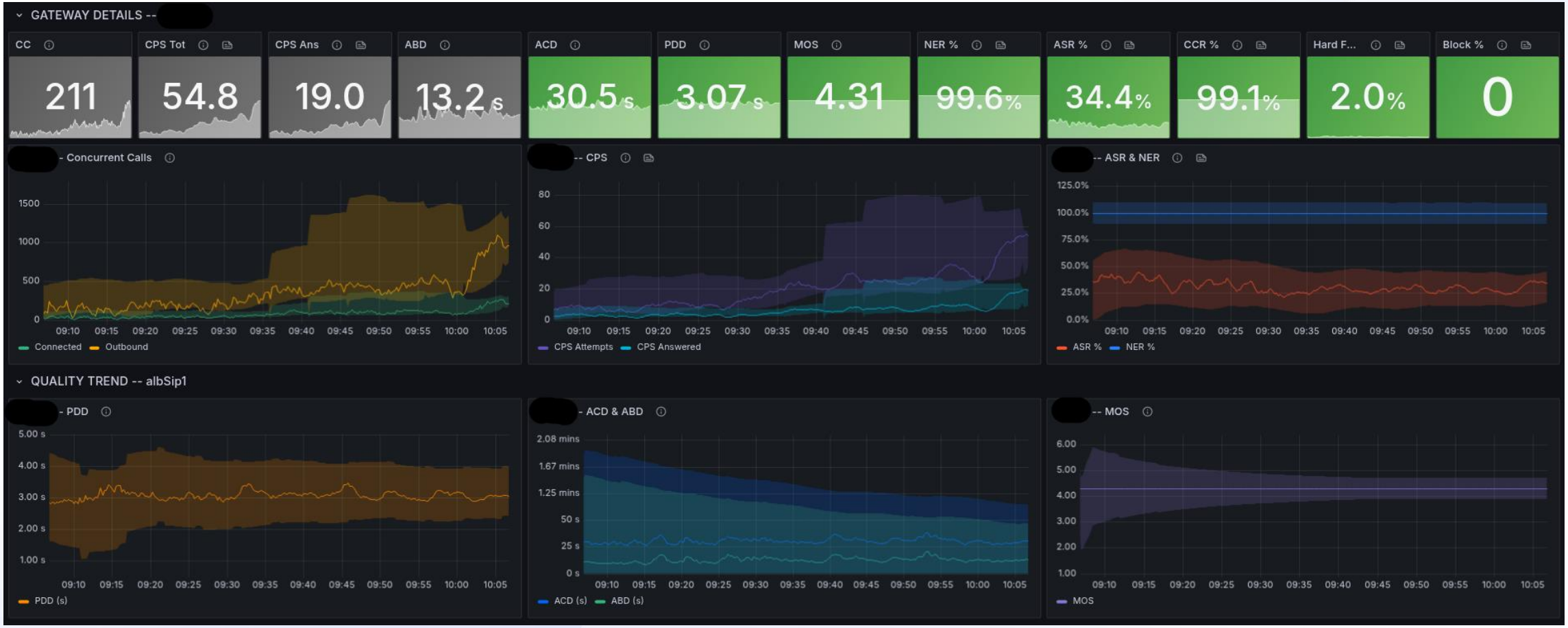


Gateway Dashboard

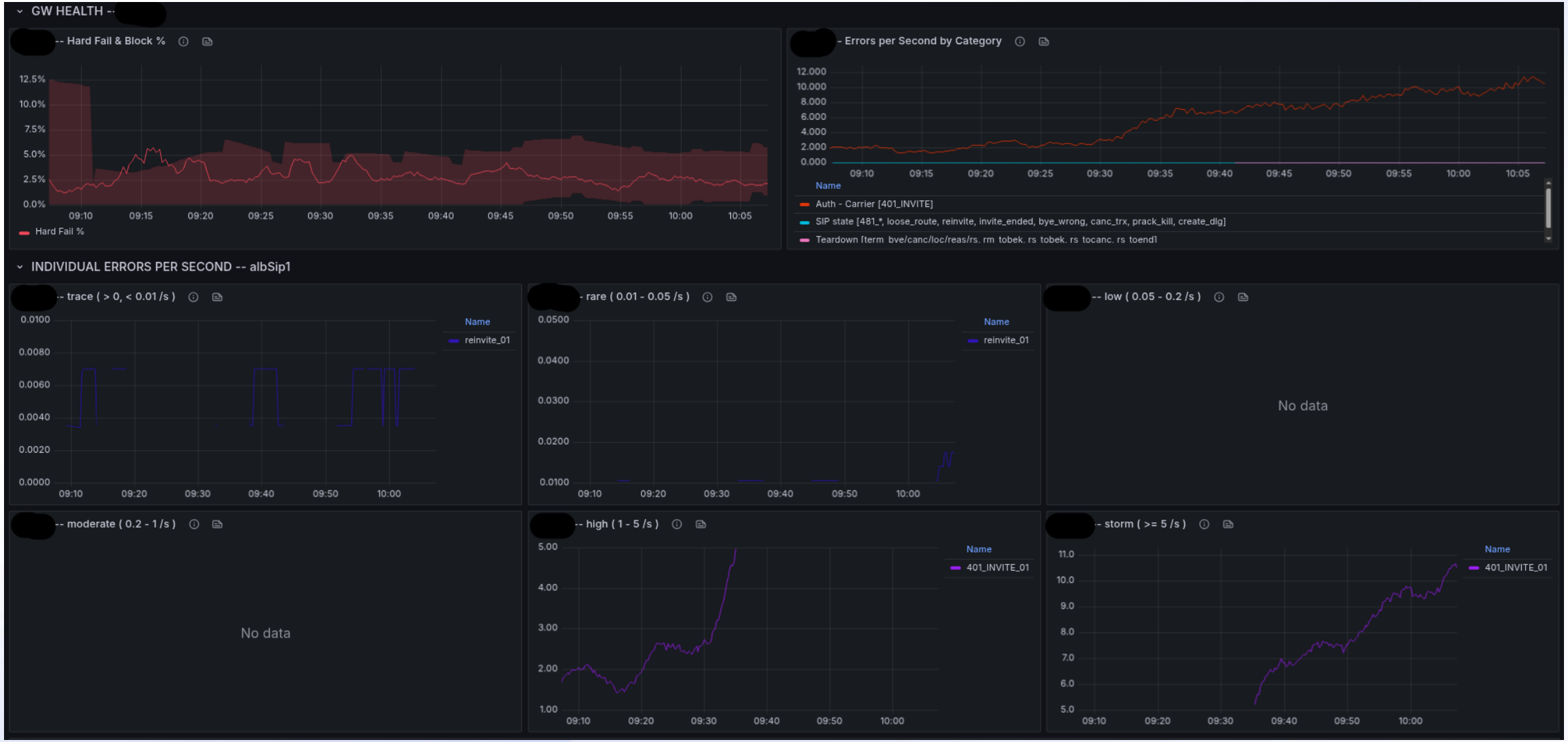


Gateway	CC Connected	CC Outbound	CPS Tot	CPS Ans	ASR %	Hard Fail %	Block %	PDD (s)	MOS
[redacted]	263	1036	57.6	18.4	33.2%	5.7%	0.0%	3.05 s	4.30
[redacted]	171	667	42.8	9.5	22.4%	39.3%	0.0%	3.23 s	4.31
[redacted]	87	299	13.2	5.3	40.1%	23.0%	80.1%	2.84 s	4.11
[redacted]	25	125	19.5	1.1	5.9%	0.1%	0.0%	3.78 s	4.32
[redacted]	214	928	49.6	16.2	34.1%	2.1%	0.0%	3.05 s	4.31
[redacted]	13	127	11.0	1.0	8.9%	35.1%	0.0%	3.11 s	3.95
[redacted]	93	603	18.8	10.0	53.5%	13.2%	0.0%	3.22 s	4.32
[redacted]	0	1	0.3	0.0	9.6%	41.0%	0.0%	1.61 s	4.35

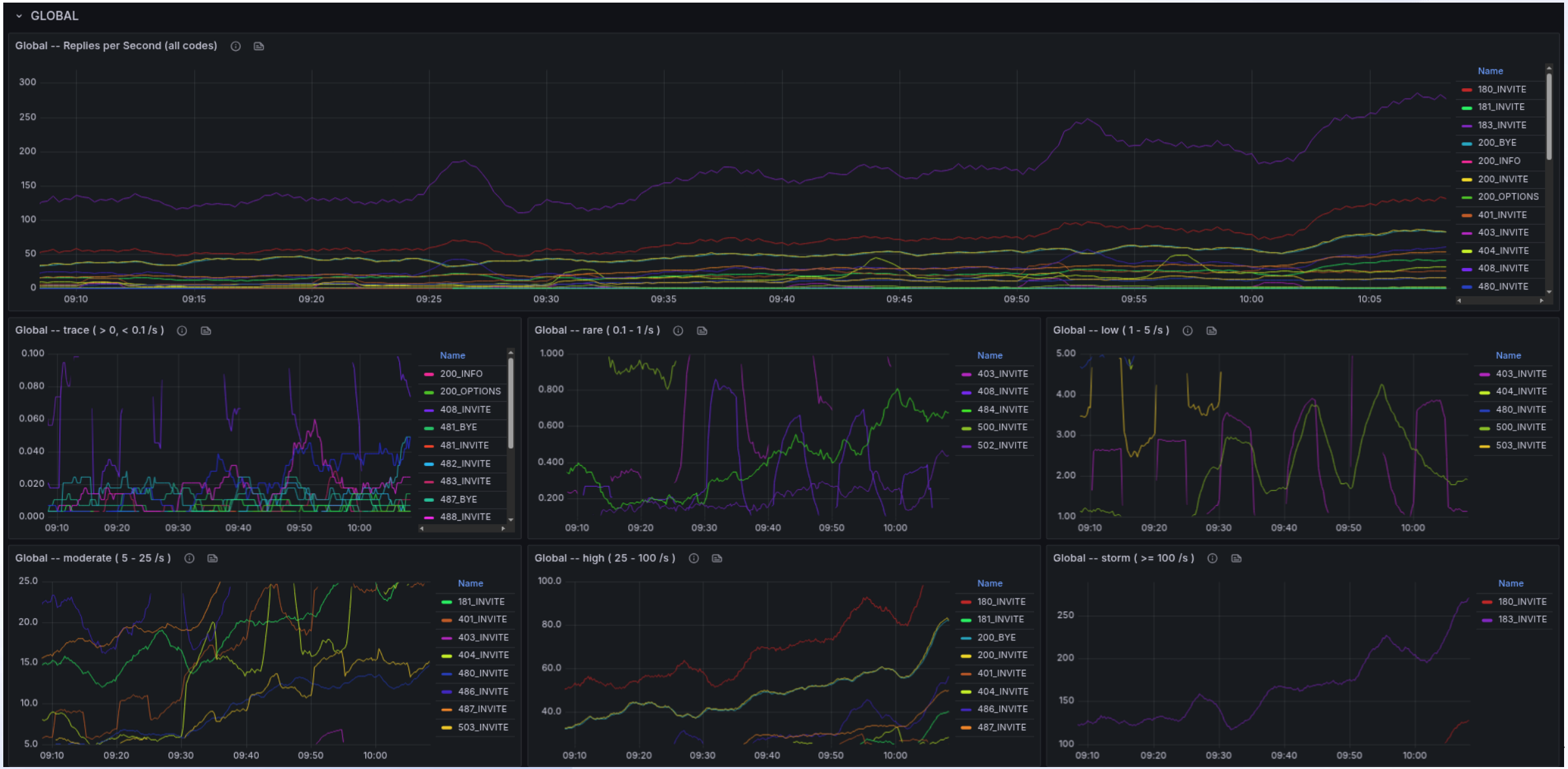
Gateway Dashboard



Gateway Dashboard



SIP Replies - General



SIP Replies – By Dimension (Gateway)

SIP Replies – By Dimension (Customer)

Anomaly Detection

Simple Alerts Fail

ALERT: cps > 1500

- Monday 10:00 - normal peak hits 1600 cps. → Alert fires. You silence it.
- Saturday 23:00 - 800 cps NOT normal. → No alert.
- Sunday noon: normal is 30 cps. Outage drops it to 0. → No alert. → The customer calls you.

- Same metric. Same threshold. Two failure modes:
  - False positive on a normal peak.
  - False negative on a real outage.

- We need a definition of **normal**

What we want are bands

A band is a per-series, per-minute envelope of 'normal'.

- upper_band(t) and lower_band(t) recomputed continuously.
- Shape follows the daily and weekly rhythm of the traffic.
  - daily ramp-up, slow-down, weekends, etc.

With bands we can:

- Visualize: overlay current value vs its expected envelope.
- Detect: anomaly = sample outside the band.
- Alert: combine breach-out-of-band with static safety rules.

One definition of 'normal' -- used by dashboards AND by alerts.

Options for Bands

There are a lot of options to do this:

- ML models -- Prophet, LSTM
  - powerful but: separate stack, continuous training, drift
  - Black (grey) box

- Vendor anomaly products -- Datadog Watchdog, Dynatrace
  - Presumably works, but...

- Statistical models in the metrics collection itself
  - lives where the metrics live -- Prometheus
  - every band is a PromQL expression you can read
  - no new service, no model lifecycle, no extra failure points

Starting Point for Anomaly Detection

There is an opensource project with this idea:

- PromQL Anomaly Detection Framework
  - Comprises a set of recording rules that generate anomaly bands for time series
  - Built by the Grafana Team
  - Similar concept used for hosted Grafana built-in anomaly detection

- Idea: use the data Prometheus already collects, bounds directly inside Prometheus with statistical means.
- No Python service. No ML pipeline. No model retraining.

- Great foundation. But not built for Telecom Traffic
  - So we rewrote it for our use case

All in Prometheus

- What we get from the PromQL Anomaly Detection Framework:
  - Proof that the approach works
  - recording rules for intermediate computations
  - change the formula, reload Prometheus, done -- zero-cost iteration
  - every band is testable, graphable

- What their stock rules do NOT handle for telecom:
  - daily AND weekly ramp-up / slow-down patterns
  - holidays (Christmas, Ferragosto, Ramadan, Easter, bank holidays)
  - per-customer and per-gateway shapes, not just global aggregates

- So we kept the stack and rewrote the math.

PromQL Anomaly Detection

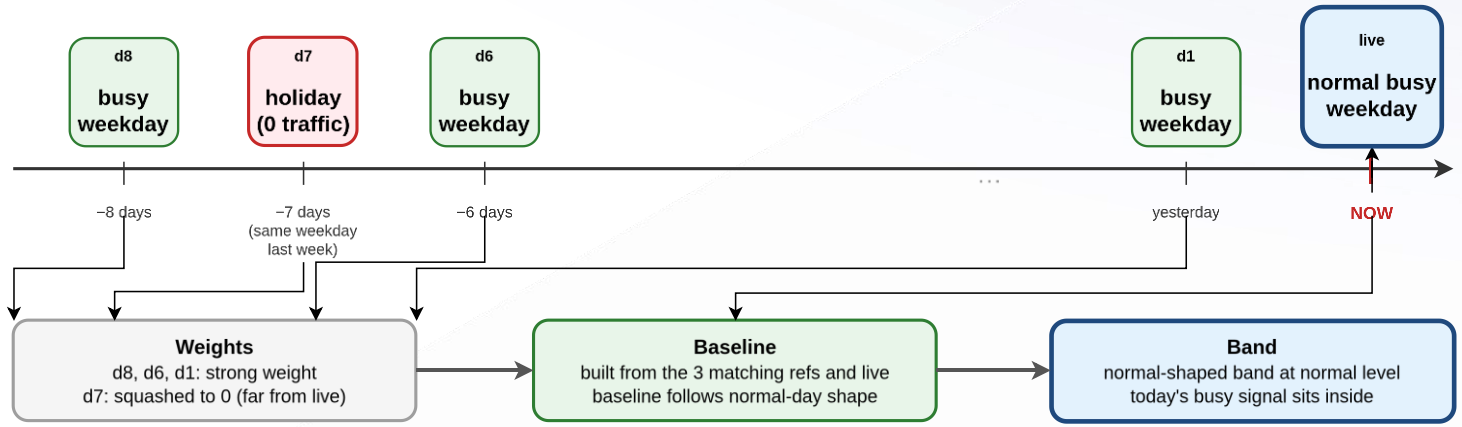
To find out if a value is normal we ask "is it moving the way similar days moved?"

- **Live signal:** What is happening right now.
- **Lookback:** We check 4 reference days (yesterday + 3 days around the same weekday last week) and see how much the metric was rising or falling at this same time of day.
- **Self-weighting:** Reference days that don't resemble today (e.g. a holiday) get automatically downweighted — the bigger the mismatch, the less influence they have.
- **Drift:** We combine those rates into a prediction of where the metric is heading next — so we catch deviations as they happen, not after.
- **Spread:** The band width auto-adjusts based on how noisy the metric is, both right now and historically. Never too tight, never too wide.
- **Result:** If the real value moves outside these bands, it's flagged as an anomaly. If no good historical match exists, the system simply tracks the live signal — no manual intervention needed.

Holiday-Awareness

Lookback on a holiday

- **Live signal** = what's happening right now.
- Combines with the window from 4 days
- One (d7) is red — holiday, nothing like today.
- So that day gets **squashed and ignored**.
- The other 3 carry the forecast.



Today is a holiday

- **None** of the 4 days will match
- Only the live signal will be used to define bands

Benefits

Holiday-aware without a calendar

- No manual holiday lists to maintain. The lookback notices if it is looking back at a holiday day.

No extra tools

- Runs entirely inside Prometheus recording rules
- No external scripts, no ML pipelines, no additional infrastructure

Testable Bands on which to base alerting

- Noisy reference days don't inflate the bands
- Four spread inputs keep bands from being too tight or too wide

formula

BASELINE

$$\text{baseline} = \text{avg_short}_{(6\,\text{min})} + \text{drift}$$

$$\text{drift} = \text{seasonal_blend}(t) - \text{seasonal_blend}(t - 6\,\text{min})$$

SEASONAL BLEND

$$\text{blend} = \frac{\sum_i w_i \cdot \text{candidate}_i}{\sum_i w_i}$$

candidates: d_1 (yesterday), $d_6/d_7/d_8$ (last week bracket)

$$w_i = \frac{1}{\left(1 + |\,\text{candidate}_i - \text{avg_short}\,|\right)^2}$$

BANDS

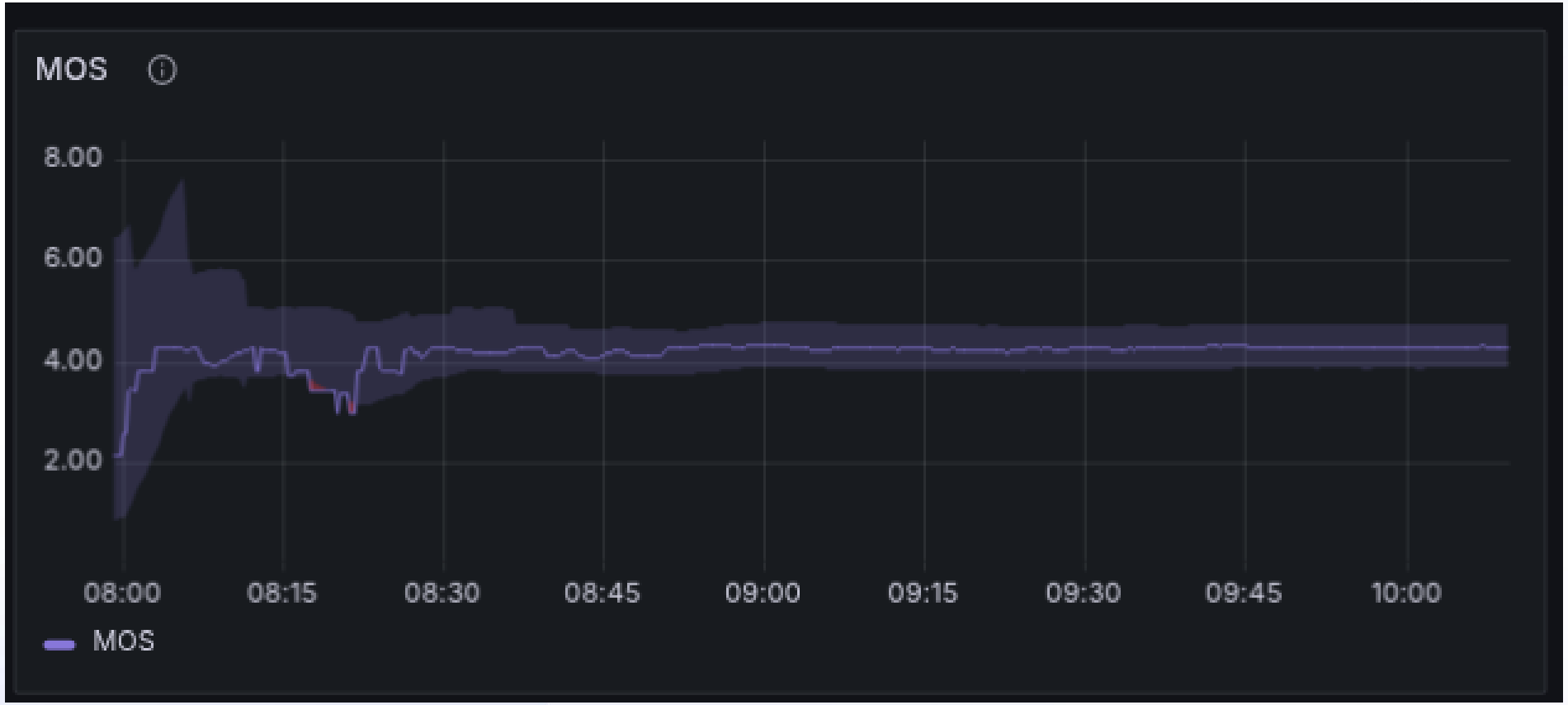
$$\text{upper} = \text{baseline} + 2 \times \text{spread}$$

$$\text{lower} = \max\left(0,\ \text{baseline} - 2 \times \text{spread}\right)$$

$$\text{spread} = \max\left(\sigma_{\text{live}},\ \sigma_{\text{seasonal}},\ 5\% \times \text{baseline},\ \text{abs_floor}\right)$$

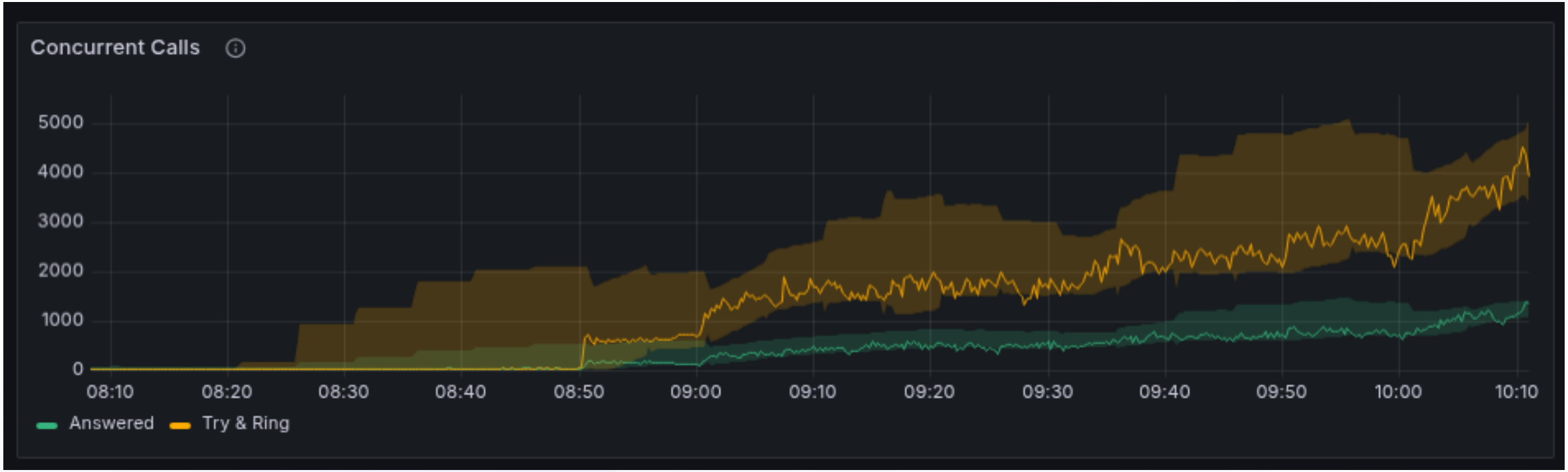
Bands

Morning Variance



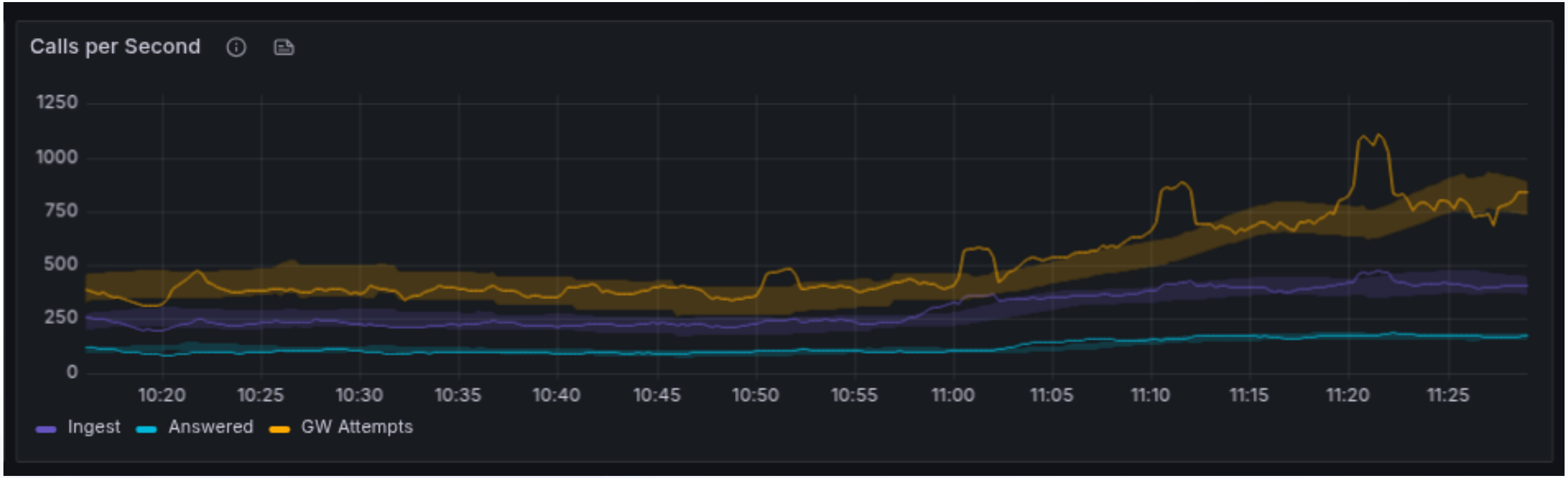
Bands

Ramp up

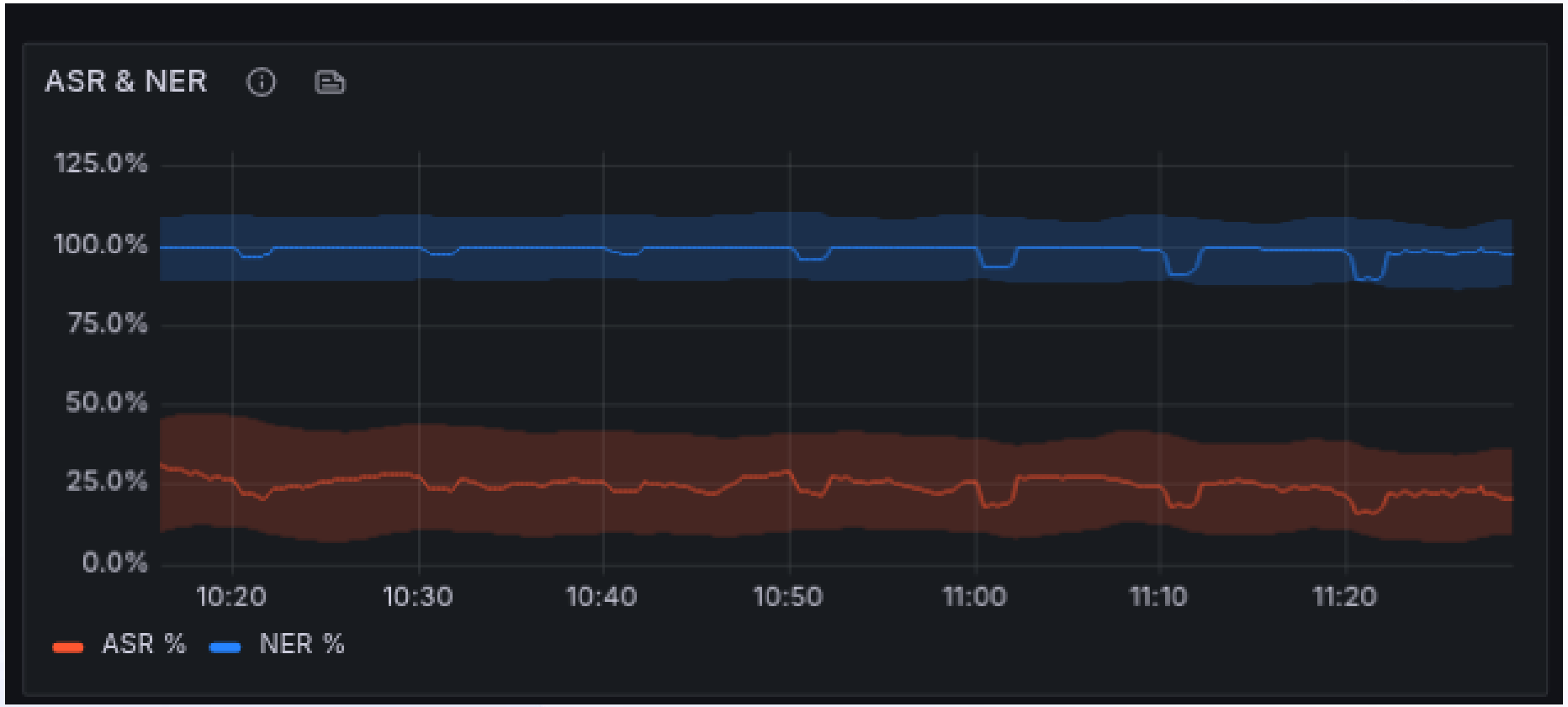


Bands

Breaches



Standard Metrics



From Anomalies to Alerts

We define what an alert is

A band breach is NECESSARY but not SUFFICIENT. We want additional gates, such as:

SIGNIFICANCE - breach of band must matter

- If the band is only barely exceeded, do we want an alert?

TRAFFIC VOLUME - ignore quiet periods

- noise on low-sample dimensions is meaningless

DURATION – breach must be sustained

- If breach holds only 2 seconds, do we want an alert?

Alerting Engine

We use Grafana-managed alerts

- Rules stored by Grafana, evaluated by Grafana
- Grafana directly manages and routes the alerts

Why Grafana-managed:

- Rules, routing, silences, contact points all in one UI
- No separate Alertmanager to deploy
- Alerts can combine queries from multiple data sources in one rule
- Version-controllable (YAML)

Grafana Alerting Features

Contact points

- Email
- Configure the SMTP settings for your email server in the `[smtp]` section of Grafana config.
- Add Emails as contact point (can customize body, subject, etc.)
- Webhook
- Give it URL you want to send the payload to

Grouping

- collapse related alerts into one notification

Timing

- wait: how long to collect before first send
- Interval: time between updates on the same group

Silences

- label-matcher based, time-bounded

From Alerts to Actions

The on-call flow, end to end

- 1. anomaly:level outside band for 3m -> AnomalyDetected fires
- 2. Alert is routed routes to Slack / …. with dashboard link
- 3. On-call clicks link -> lands on pre-filtered Grafana panel
- 4. Band overlay shows ACTUAL vs EXPECTED at one glance
- 5. Drill down by dimension (GW / CU / RU) to find the breaching entity

- Typical alert-to-root-cause time: under a minute.

Thank you

info@opentelecom.it

code:
https://github.com/gmaruzz/opensips_summit_2026